UNIVERSITY OF RUHUNA

FACULTY OF ENGINEERING

ServiceSync

Service Booking Platform

Web Development Module
Individual Mini Project Report

ANUSIKAN S.

EG/2023/5480

29 October 2026

Abstract

ServiceSync is a web application that connects customers who need service work with providers who can complete it. Customers post jobs, compare quotes and select a provider. Providers browse relevant jobs, submit offers and track bookings, schedules, reviews and earnings. The application uses React for the interface, Express for the REST API, MongoDB for persistent records and Cloudinary for images.

The main project entity is the job, with complete Create, Read, Update and Delete operations. Authentication, role and ownership checks protect workflow actions. Quote acceptance preserves an agreed price and creates an accepted booking; completion moves the booking to history, where a review is optional. Automated tests and guided manual checks support the implementation. The project is demonstrated locally; public hosting has not been completed.

Report organisation

Pages 2 to 3 explain the problem, architecture and database. Pages 4 to 8 describe CRUD and the user interfaces. Page 9 records validation and testing. Page 10 provides setup instructions, limitations, conclusions and references.

1 Project Overview and Architecture

Problem and objectives

Service requests handled through separate calls and messages make it difficult to compare offers and retain a consistent record of work. ServiceSync brings requests, provider quotations, booking decisions and communication into one workflow. The objective is to provide a usable service marketplace with persistent job CRUD and clear separation between customer and provider responsibilities.

Customers can manage job descriptions, budgets, locations and appointment times, inspect provider profiles, accept offers, communicate and review completed work. Providers can filter opportunities, quote a price, monitor accepted work and inspect earnings and feedback. Search, sorting, image viewing and a mobile navigation menu support day-to-day use.

Architecture and implementation

Layer	Technology	Responsibility
Presentation	React 19, React Router, Vite	Role-specific pages, reusable forms and cards, API calls and responsive layout
Application	Node.js, Express 5	REST routes, JWT authentication, access checks and workflow services
Persistence	MongoDB, Mongoose	Seven document collections, references, validation and quote-acceptance transactions
Media and reporting	Cloudinary, Recharts	Image storage and visual summaries of completed booking earnings

The browser sends HTTP requests through client/src/services/api.js. Express routes validate the request and use models and reusable services to read or update MongoDB. Passwords are hashed using bcrypt. Uploaded files are handled with Multer and Cloudinary storage. Images are displayed from stored URLs, with an in-app viewer for job photos.

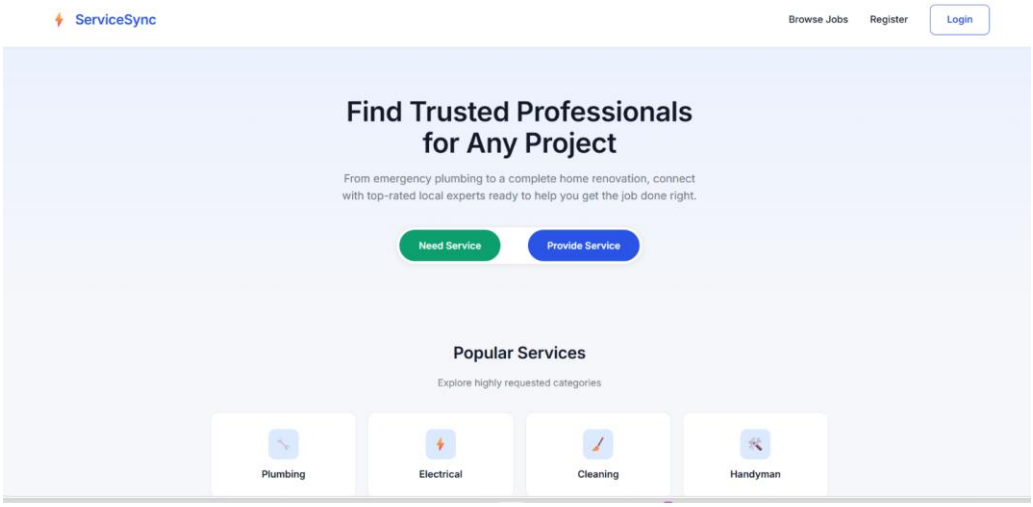


Figure 1 Public homepage introducing the service booking platform

2 Database Design and Relationships

The database contains User, Job, Quote, Booking, Review, Message and Notification collections. Each document has an ObjectId identifier and timestamps. Figure 2 shows the main workflow relationships and the reference fields that connect the remaining collections. References are application-managed links, rather than relational database foreign-key constraints.

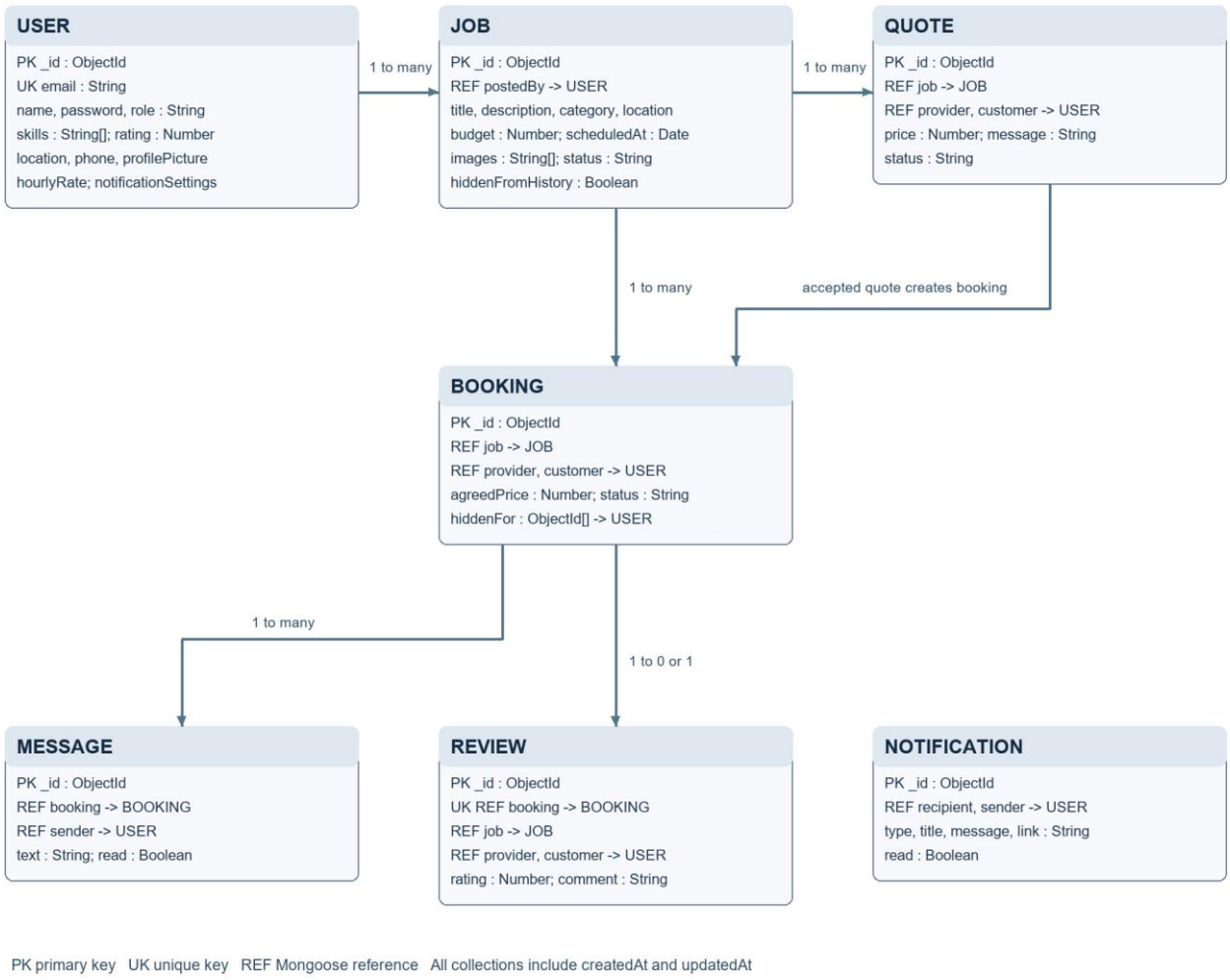


Figure 2 Entity relationship diagram with main workflow links and explicit document references

Integrity and historical records

User.email has a unique index. Review.booking is unique, allowing at most one review for a booking. A job can receive multiple quotes and requests, while selection assigns it to one provider through guarded acceptance. A booking may have many messages and zero or one review. Customer and provider fields refer to User documents with the relevant role.

Job status progresses through open, assigned and completed; deleting an open job sets its status to deleted. Booking status is pending, accepted, declined or completed. Quote status is pending, accepted, declined or withdrawn. An agreed booking price is retained separately from the editable job budget. Hiding history uses hiddenFromHistory on jobs or hiddenFor on bookings, preserving reviews, linked records and provider earnings.

3 Job CRUD Implementation

Jobs provide the full CRUD entity required by the evaluation rubric. A customer posts a description, category, location, budget and optional appointment, with up to three photos. Providers read the available jobs and can search by title, filter by category, location and budget, and sort by date or price. Provider results default to a skills and location match ranking.

Operation	REST endpoint	Behaviour
Create	POST /api/jobs	Validate form data and persist the customer-owned job
Read	GET /api/jobs; /api/jobs/mine	Browse open jobs or list the current customer's visible jobs
Update	PUT /api/jobs/:id	Owner edits an open job; rejected input is not saved
Delete	DELETE /api/jobs/:id	Owner confirms removal; soft-delete and close pending offers

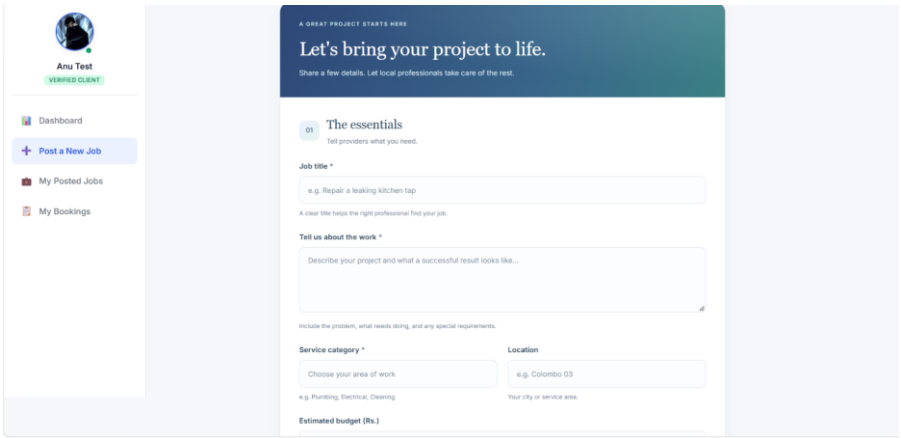


Figure 3 Customer form for creating a service request

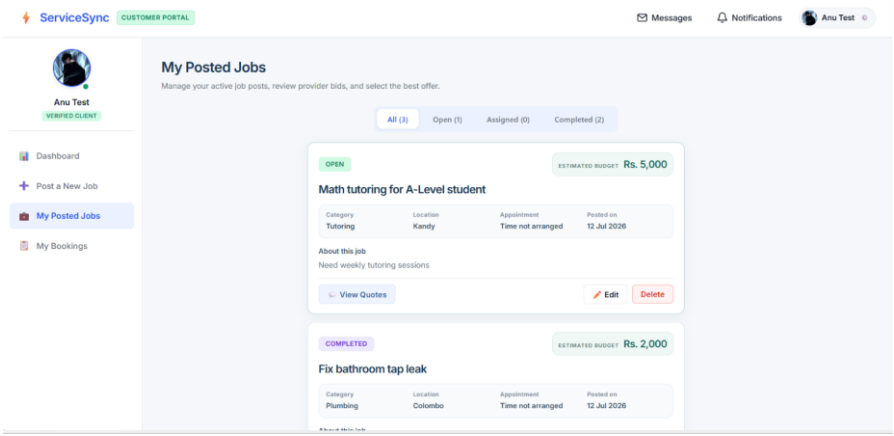


Figure 4 Posted job cards with details and management actions

Edit and delete operations check ownership and job status on the server. Completed cards provide View booking and Remove from history. History removal hides the customer listing without deleting the completed booking or changing its agreed price.

4 Quotes Bookings and Communication

A provider opens an available job, inspects its description and photos, and submits a positive price with an optional message. Customers inspect offers through View Quotes and can open a provider profile before deciding. A count badge indicates pending quotes. A provider can withdraw a pending offer; the customer can decline or accept it.

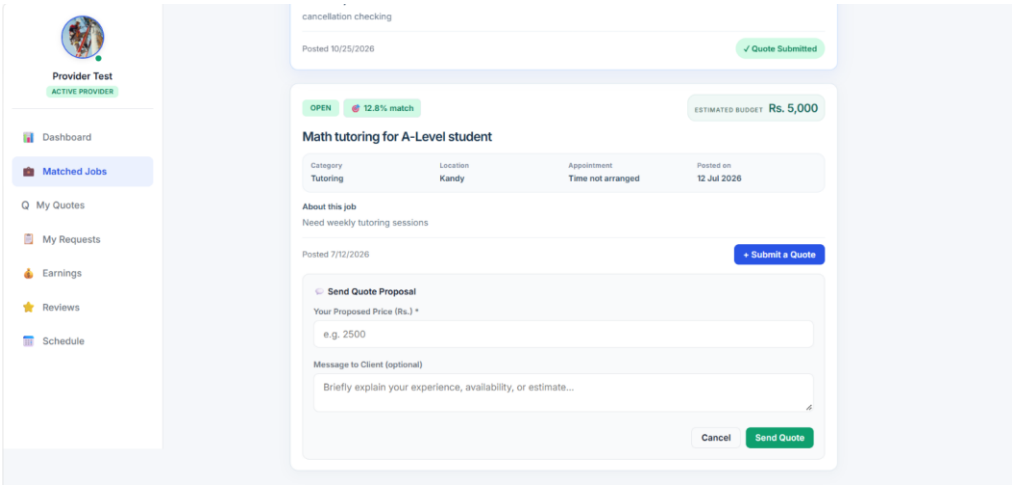


Figure 5 Provider quotation form attached to an available job

Accepting a quote executes a MongoDB transaction that claims the open job, accepts the selected quote, creates an accepted booking with the quoted price and declines competing pending offers. Guarded updates prevent competing acceptance actions from assigning the same job twice. The API also supports direct provider requests at the job budget; existing pending requests remain manageable in My Bookings.

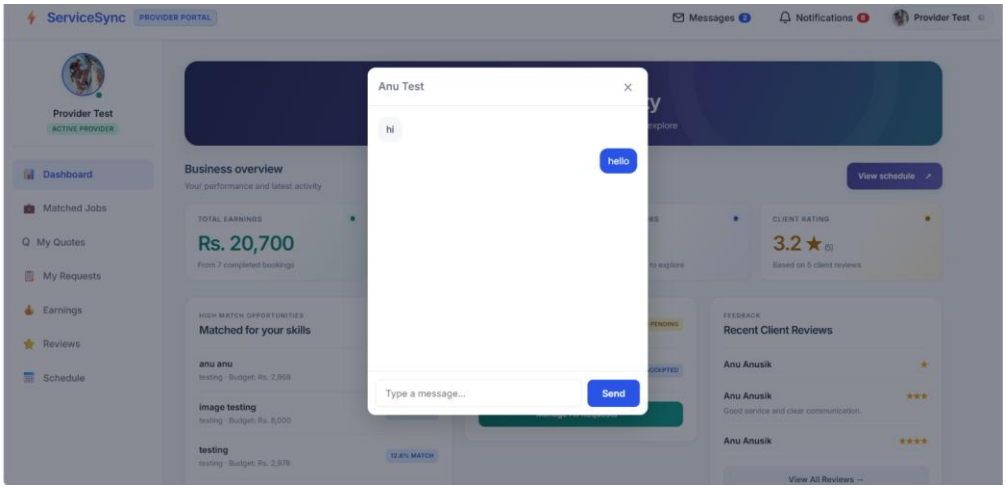


Figure 6 Booking conversation displayed inside the application

Only the customer and provider belonging to a booking can read or send messages in its thread. Saved messages retain their order and read state. Messages and unread chat counts are separate from the booking and quote notification bell. The customer completes accepted work; completed and declined records appear immediately in booking History.

5 Customer and Provider Portals

Separate dashboards summarise the information relevant to each role. Customer navigation provides Dashboard, Post a New Job, My Posted Jobs and My Bookings. Provider navigation includes Matched Jobs, My Quotes, My Requests, Earnings, Reviews and Schedule. Public profiles expose professional details and feedback without exposing passwords.

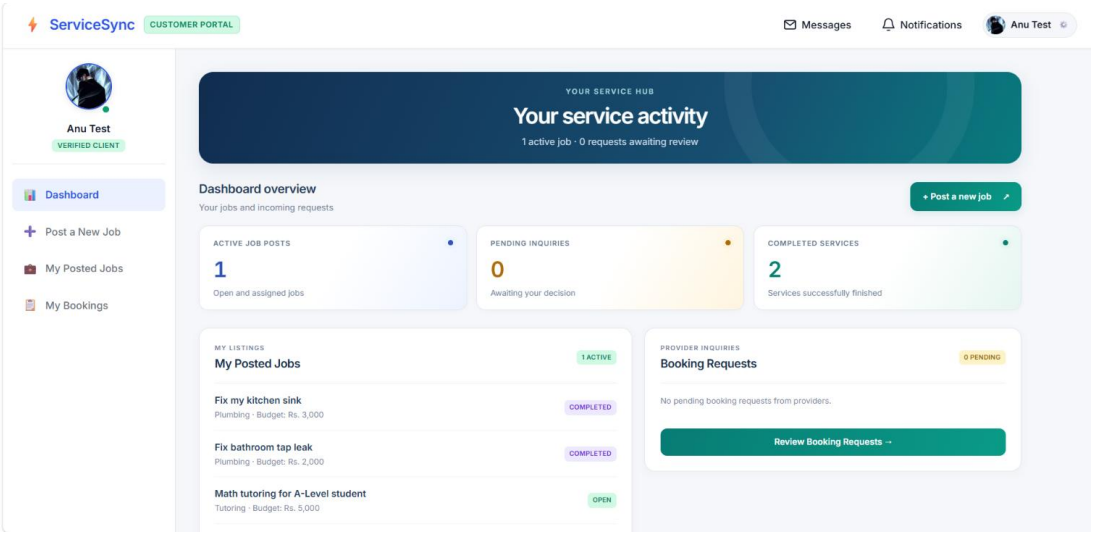


Figure 7 Customer dashboard summarising posted jobs and booking requests

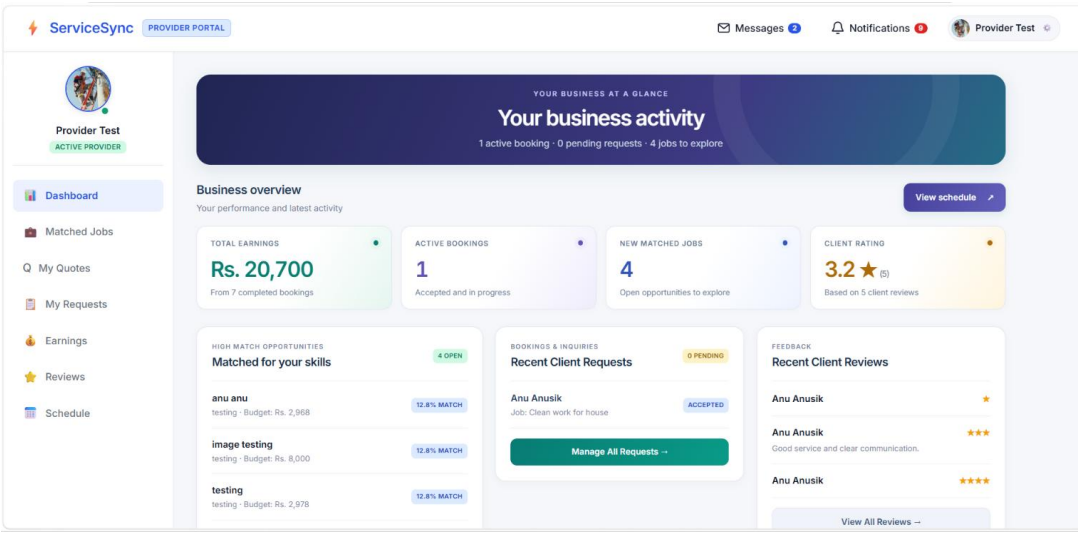


Figure 8 Provider dashboard summarising work opportunities and activity

Navigation and notifications

The desktop portals use a persistent sidebar and a shared top navigation bar. On smaller screens, a three-line dropdown replaces the sidebar; Messages and Notifications use icons with count badges and the profile remains accessible through an avatar. Booking and quote preferences are saved per account. A refresh or return to the jobs page updates pending quote counts without requiring every quote panel to be opened.

6 Reviews History and Earnings

Reviews are optional and available only after the customer completes an accepted booking. Completed work moves to My Bookings History even when no review is submitted. Review opens the form, Maybe later closes it and successful submission changes the action to Reviewed. The server checks customer ownership, completion status and the unique booking-review relationship.

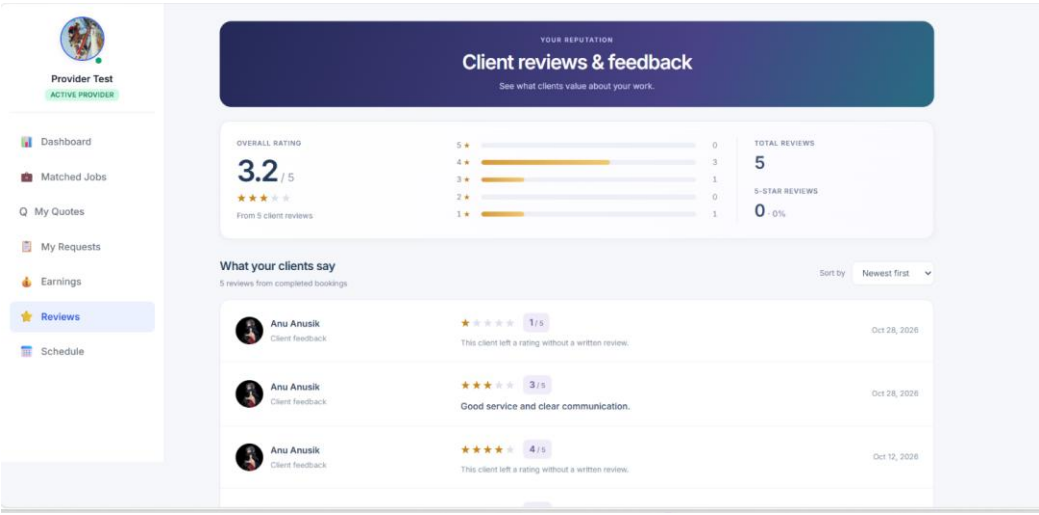


Figure 9 Provider review summary and customer feedback

Provider ratings are calculated from saved reviews. Earnings summarise agreed prices for completed bookings, rather than pending quotes or open job budgets. The figures are reporting totals and do not represent online payment processing. Prices for older records can be resolved from accepted quotes without rewriting their history.

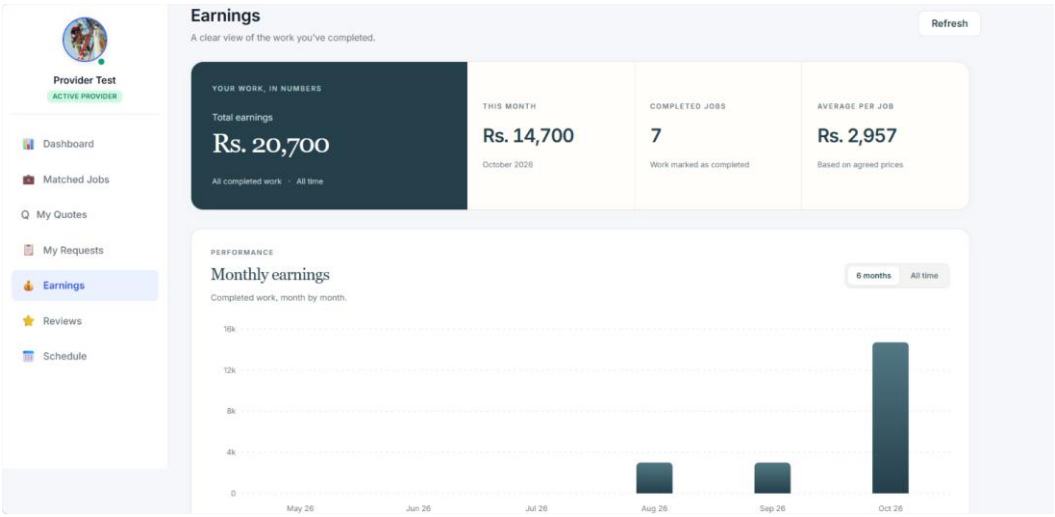


Figure 10 Earnings summary and chart for completed work

Removing a completed job from My Posted Jobs or a finished booking from History changes visibility only. The other participant retains their booking, and reviews, completion dates and provider earnings remain available. Customers do not have to leave feedback before removing their own history card.

7 Scheduling Images and Mobile Support

Customers may select a future appointment when posting or editing an open job. Appointments are stored as dates and displayed consistently in Sri Lanka time. Accepted bookings appear in the provider schedule. Where no appointment is entered, the interface indicates that a time has not been arranged.

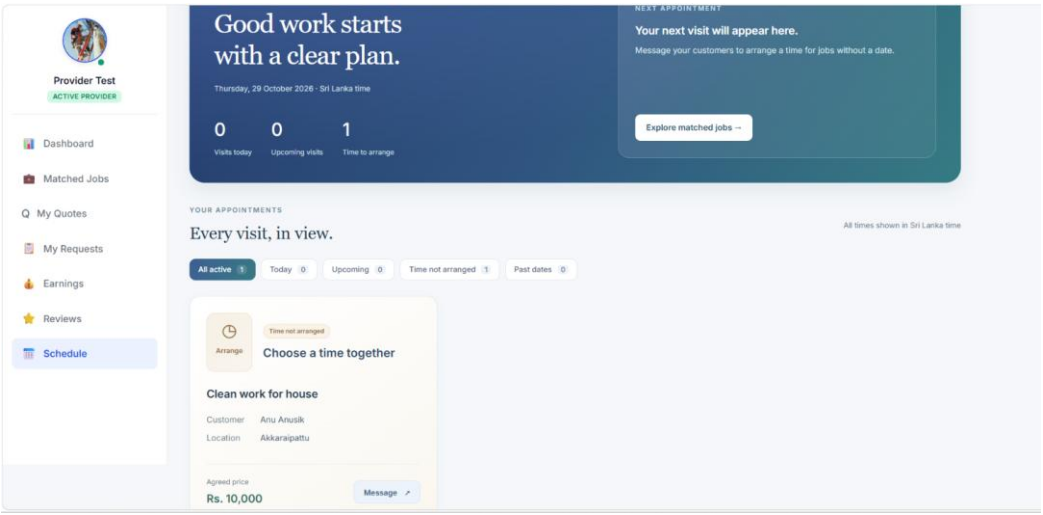


Figure 11 Provider schedule showing accepted work and appointment information

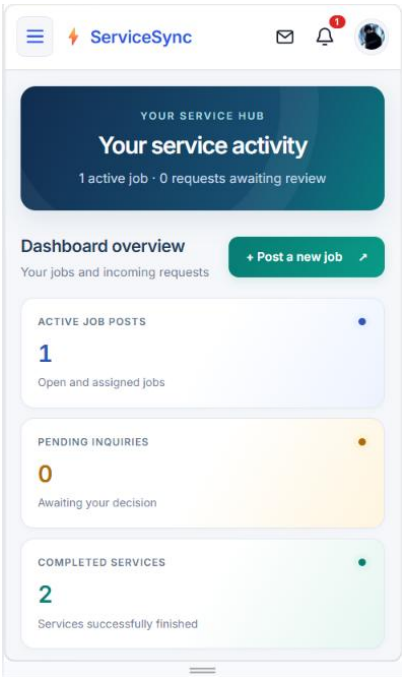


Figure 12 Responsive customer dashboard and compact mobile navigation

Job photos are uploaded to Cloudinary and retained as URLs in the job document. Both the owner and providers can inspect thumbnails. The in-app viewer offers full-size viewing, Previous and Next controls, keyboard navigation and dismissal without leaving the website. Responsive grids and wrapping controls keep job details, quote actions, booking history and chat usable on narrow screens.

8 Validation Access Control and Testing

Validation and error handling

Job creation and editing validate titles, descriptions, categories and finite nonnegative budgets on both client and server. Budget filters reject invalid numbers and reversed ranges. Appointment validation rejects malformed or past dates while allowing an unchanged existing appointment. Registration validates a trimmed name, email format, role and optional provider fields. Passwords require at least six non-padding characters and respect bcrypt's 72-byte input limit.

JWT middleware protects API routes. Role checks distinguish customer and provider actions, and ownership checks protect jobs, quotes and bookings. Message routes restrict access to booking participants. Client route guards improve navigation, while server checks enforce authorization. These controls support the demonstrated workflow; they are not a substitute for a production security assessment.

Automated verification on 29 October 2026

Check	Result	Coverage or observation
Server Jest suite	170 tests passed 20 suites passed	Roles, prices, quote acceptance, transactions, notifications, appointments, job history and deletion
Frontend tests	6 passed	Notification preferences, alert selection and optional sound helpers
ESLint	Passed	No lint errors or warnings reported
Vite production build	Passed	Bundle created; JavaScript chunk-size warning remains

Manual and live workflow verification

Guided manual checks were completed for account registration and login; job creation, editing and deletion; provider search, filtering and sorting; quote acceptance; messaging; completion and reviews; server-restart persistence; role-based page access; notifications; scheduling; mobile navigation; and image upload/viewing. These checks provide functional evidence for the demonstrated scenarios, rather than exhaustive coverage of all devices or input combinations.

A separate live verification script used synthetic accounts in an isolated MongoDB database. It exercised HTTP routes for job CRUD, filtering, quote acceptance, agreed prices, messages, completion and review, then disconnected and reconnected to verify persisted job states. Normal application records were not modified by that check. The automated tests mainly isolate application logic using model mocks, so the live check complements rather than replaces them.

Repeatable commands

Server: npm test -- --runInBand
Client: npm test; npm run lint; npm run build
Optional live verification: cd server, then npm run verify:live. This creates a separate verification database and requires permission to create it.

9 Setup Limitations and Conclusion

Installation and local execution

Clone the GitHub repository and use a current supported Node.js runtime compatible with Vite, such as Node 22.12 or later. Run npm ci separately in server and client. MongoDB must be a replica set or an Atlas deployment because quote acceptance uses transactions. Cloudinary credentials are required for uploads.

Location	Configuration or command
server/.env	MONGO_URI, JWT_SECRET, PORT and CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET
client/.env	Optional VITE_API_BASE_URL; local default is http://localhost:5000/api
Backend terminal	cd server, then npm run dev
Frontend terminal	cd client, then npm run dev; open the local URL printed by Vite

Register one customer and one provider account to demonstrate the complete workflow. Keep .env files and secrets outside version control. The repository separates React pages, components and utilities from Express routes, Mongoose models, middleware and workflow services. Git records code changes, and the README documents installation and feature behaviour.

Limitations and future development

Public hosting is not complete; the application is currently demonstrated locally. There is no online payment gateway, administrator portal or password recovery workflow. Earnings are completed-booking summaries. Large job lists would benefit from server-side pagination and database-level sorting; the current browse service sorts returned results in application memory. Route-level code splitting could reduce the production bundle warning. Deployment also needs environment-specific origin restrictions, monitoring and broader security and browser testing.

Conclusion

ServiceSync implements persistent job CRUD and connects it to provider quotations, bookings, messaging, scheduling and optional reviews. Role and ownership checks protect the principal actions, while history hiding preserves completed records and earnings. The screenshots, automated checks and manual workflow tests demonstrate the project's implemented functionality and its remaining deployment limitations.

References

Project repository: https://github.com/anuanusikan/Smart_service_Booking_Platform

Course reference: Mini Project Evaluation Rubric COM, EC4307 Web Application Development, 14 October 2026.

Framework documentation: React https://react.dev/learn; Vite https://vite.dev/guide/; Express https://expressjs.com/; Mongoose https://mongoosejs.com/docs/.

Supporting documentation: MongoDB https://www.mongodb.com/docs/; Cloudinary https://cloudinary.com/documentation; React Router https://reactrouter.com/; Recharts https://recharts.github.io/.

Implementation assistance: AI tools supported code review, selected implementation changes, testing guidance and report preparation. The submitted implementation and report require author review under the module's academic-integrity policy.